



École Nationale Supérieure d'Informatique pour l'Industrie et l'Entreprise

Rapport de projet COAV

Vérification statique des séquences d'appels aux fonctions
collectives MPI

Jules Hermier & Simon Bélier

Date: 21 Novembre 2025

Contents

I	Introduction	3
II	Création du plugin GCC	3
	II.1 Création d'une passe	3
	II.2 CFG	4
	II.3 Détection des collectives	5
	II.4 Calcul de l'ordre d'appel des collectives	6
	II.5 Repérage du deadlock	7
	II.5.a Post dominance	7
	II.5.b Frontière de post-dominance (PDF)	8
	II.5.b.a Noeud	8
	II.5.b.b Ensemble de noeud	9
	II.6 Résultat du plugin	10
III	Conclusion / pistes d'amélioration	11

I Introduction

MPI ou *Message Passing Interface* est un standard pour définir des programmes pour des architectures parallèles. Elle définit une bibliothèque de fonctions utilisable dans plusieurs langages.

Une problématique principale de la programmation MPI est la synchronisation: pour réaliser des calculs ou des opérations, les programmes devront s'attendre pour la mise en commun de données. Ces moments ont lieu lors de l'appel de *collectives* comme par exemple MPI_BARRIER (arrêt du processus MPI tant que tous les processus n'ont pas atteint une collective MPI_BARRIER).

Si les conditions d'appels des collectives ne sont pas remplies, les processus peuvent se retrouver bloqué indéfiniment, causant un *deadlock*. Ces arrêts peuvent être dûs à anticiper empiriquement dans un programme de taille conséquente, ainsi il convient de programmer d'une manière à éviter leur présence en amont.

Pour cela, nous proposons d'implémenter un plugin pour le compilateur GCC analysant chaque chemin du graphe de contrôle d'exécution, émettant un avertissement si des chemins d'exécution contiennent des séquences d'appels MPI différentes, causes d'un potentiel deadlock.

II Création du plugin GCC

II.1 Création d'une passe

Un compilateur est un outil transformant du code haut-langage dans un langage machine. Pour cela, il convertit le code dans une représentation intermédiaire sur laquelle il exécute plusieurs transformations (passes) pour en faciliter l'analyse et l'optimiser par la suite sans en changer l'exécution.

Les passes peuvent modifier le graphe de flot de contrôle pour l'optimiser, l'étendre, le réduire ou pour fournir des avertissements à l'utilisateur.

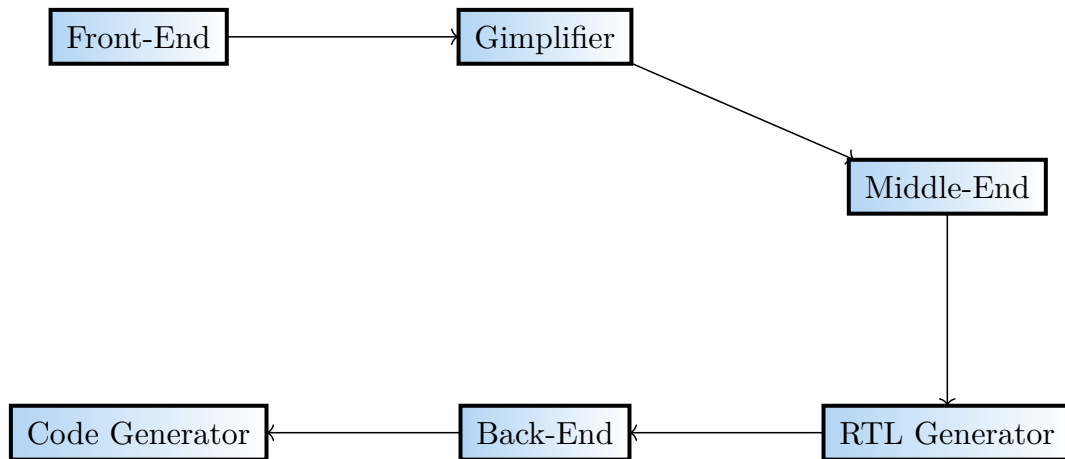


Figure 1: schéma du fonctionnement de GCC

GCC fournit une interface permettant d'insérer des passes manuellement sous forme de plugins. La création d'une passe passe par plusieurs étapes:

- La création d'une classe héritant du type de la passe souhaitée
- Surcharge des trois méthodes principales d'une passe:
 - *clone*: uniquement nécessaire si la passe est créée via un plugin.
 - *gate*: Une fonction booléenne décidant de si la passe doit être appliquée sur la fonction actuelle.
 - *execute*: La logique de modification de la passe, uniquement appelée si *gate* renvoie une valeur positive.
- création d'un objet contenant les informations relative à la passe (son type, son nom, où l'insérer dans le processus de compilation, etc...)
- une fonction *plugin_init* qui sera le point d'entrée du plugin lors du chargement de GCC, permettant d'ajouter la passe au procédé de compilation.

II.2 CFG

Un graphe de flot de contrôle ou CFG est un multi-graphe orienté tel que:

- Les noeuds sont les blocs de base
- Les arcs représentent les ordres possible d'exécution

Le noeud de départ correspond à la première instruction du programme, et il peut y avoir plusieurs noeuds finaux selon les différents points de sortie.

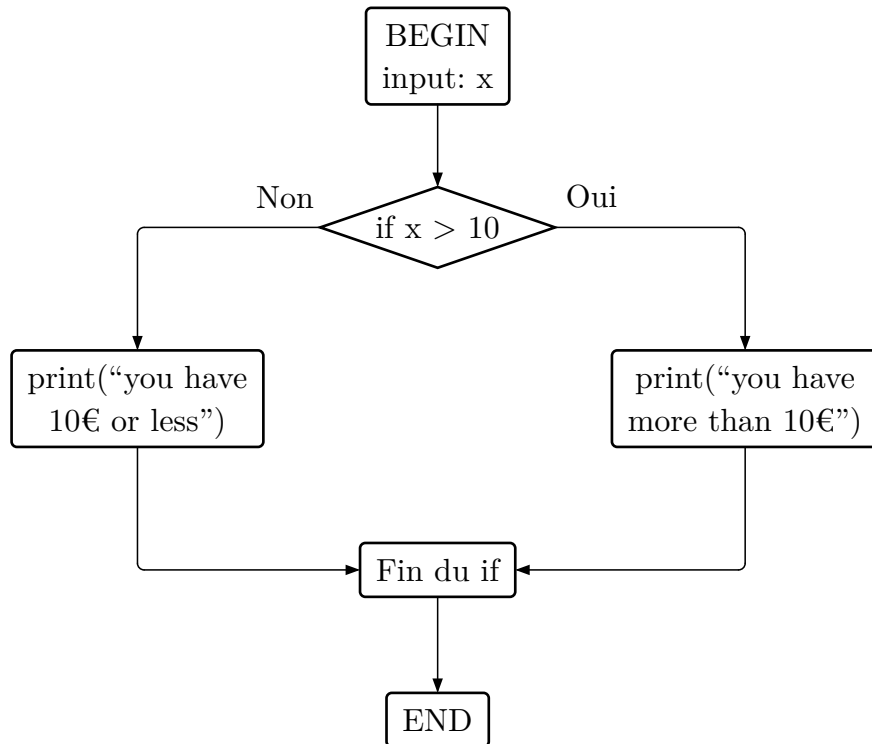


Figure 2: Exemple de graphe de flot de contrôle ou CFG

En pratique, le CFG correspond aux différents chemins d'exécutions que peut prendre un programme en temps réel, ainsi raisonner dessus peut nous permettre de détecter des deadlocks ou des erreurs de logique.

Dans le cadre de GCC, le CFG est composé de *basic blocks*, information sur une partie de code séquentielle avec un seul point d'entrée et de sortie. Ils contiennent toutes les informations sur le code, les fonctions appelées et les différents branchements depuis une instruction.

II.3 Detection des collectives

Pour détecter un possible deadlock MPI il est important d'identifier ces dernières dans le CFG. Pour le faire, nous avons besoin de 3 choses :

- le nom des collectives/ leur définition définit dans `MPI_collective.def`,
- pouvoir récupérer la fonction lié à un *basic block* du cfg,
- pouvoir itérer sur l'ensemble du cfg.

Heureusement pour nous, les sources de GCC nous permettent de faire tout cela. Ainsi nous parcourons les *basic block* du cfg de la fonction courante avec `FOR_EACH_BB_FN(bb, cfun)`. Pour récupérer le fonction MPI lié à ce *basic block* il est nécessaire d'itérer sur ses *statement* et de trouver si un correspond à une collective MPI (fonction donné en TP `get_mpi_call_code(gimple* stmt)` dans `mpi_collective.h`).

```

6 void mpi_call(int c)
7 {
8     MPI_Barrier(MPI_COMM_WORLD);
9
10    if(c<10)
11    {
12        printf("je suis dans le if (c=%d)\n", c);
13        MPI_Barrier(MPI_COMM_WORLD);
14        MPI_Barrier(MPI_COMM_WORLD);
15    }
16    else
17    {
18        printf("je suis dans le else (c=%d)\n", c);
19        MPI_Barrier(MPI_COMM_WORLD);
20    }
21 }
22

```

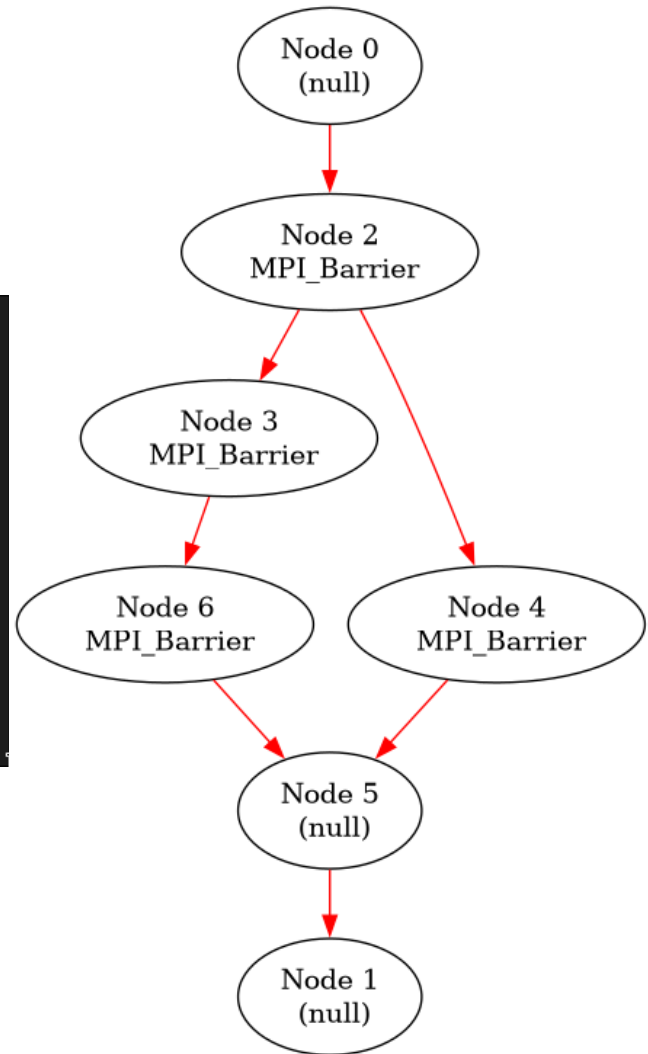


Figure 3: Exemple de conversion de code vers CFG
capturant les collectives MPI

II.4 Calcul de l'ordre d'appel des collectives

A présent nous pouvons facilement déterminer quels noeuds du cfg est sont une collective mpi. Le tout est maintenant de les stocker dans une structure qui permet de vérifier les deadlock via la *frontière de post dominance* (voir Section II.5).

Nous devons stocker ensemble les collectives de même rang (notion définit au prochain paragraphe) et même code (MPI_Reduce, MPI_Barrier, ...). Nous avons donc fait le choix de stocker cela dans une `map<rang, map<code , set<int>>>` avec `set<int>` l'ensemble des index des *basic block* de même nature et de même rang.

rang↓/code→	MPI_Init	MPI_Barrier	MPI_Reduce	MPI_AllReduce	MPI_Finalize
1	{}	{2}	{}	{}	{}
2	{}	{3,4}	{}	{}	{}
3	{}	{6}	{}	{}	{}

Table 1: Stockage des collectives MPI de la Figure 3

Afin de déterminer le rang, l'ordre d'appel des différents collectives, nous retirons les arrêtes créant des boucles dans le CFG afin d'éviter une boucle infinie dans l'attribution des tailles de rang. Pour cela nous utilisons la fonctionnalité de GCC permettant de marquer les "back-edges" du CFG pour créer un nouveau graphe sans ces arrêtes dans une bitmap que nous utiliserons pour les futures opérations.

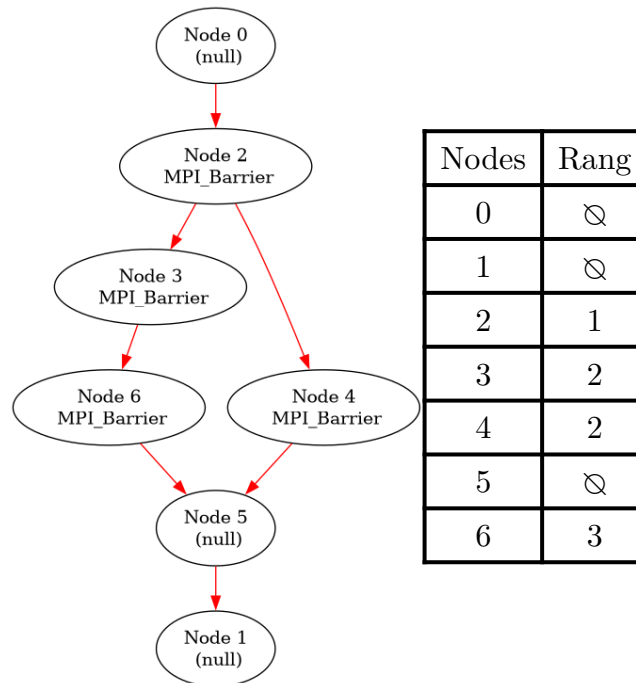


Figure 4: Exemple de calcul de rang

II.5 Repérage du deadlock

Il reste à déterminer les conditions pour un deadlock potentiel: il faut vérifier qu'il n'existe pas de chemins tels que les séquences d'appels MPI sont différents. Pour cela, on introduit la notion de post-dominance.

II.5.a Post dominance

Un nœud X post domine un nœud Y si tous les chemins de Y aux puits passent par Y.

$$\text{PDom}(Y) = \left(\bigcap_{p \in \text{succs}(Y)} \text{PDom}(p) \right)$$

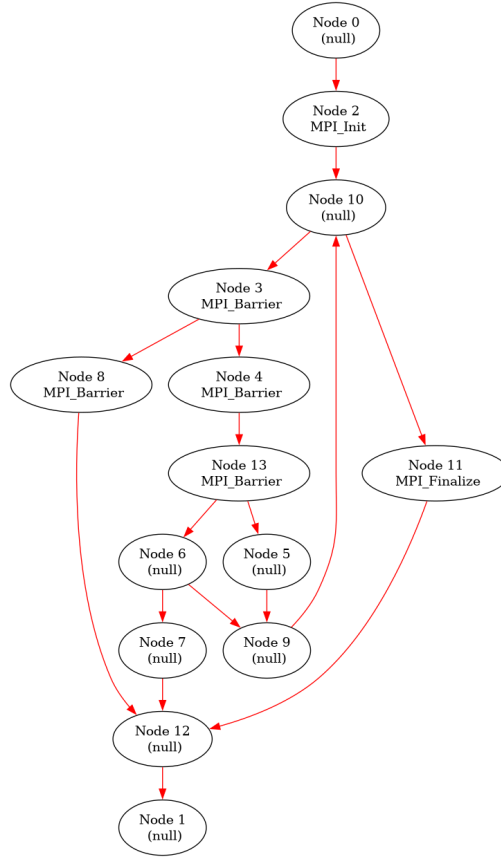
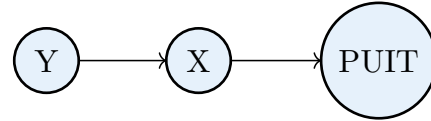


Figure 5: Ici le noeud 13 post domine 4

II.5.b Frontière de post-dominance (PDF)

II.5.b.a Noeud

À partir de la notion de post-dominance on peut définir la frontière de post dominance pour un noeud comme:

$$\text{PDF}(y) = \{x \mid \exists Sx \in \text{succs}(x), \text{ tel que } y \gg_p Sx \text{ et } y \not\gg_p x\}$$

Cette notion permet de s'assurer que l'on exécute bien un morceau de code peu importe le chemin pris et aussi que les collectives soient appelées dans le même ordre. La frontière de post-dominance d'un noeud correspond intuitivement à l'ensemble minimal de noeuds du cfg où l'effet de la post-dominance s'arrête. Ce sont les noeuds pour lesquels un successeur est post-dominé par y, mais pas le noeud lui-même. Elle constitue la limite entre les régions du programme entièrement contrôlées par y et celles qui ne le sont pas.

II.5.b.b Ensemble de noeud

La notion de frontière de post-dominance pour un ensemble V est intuitivement similaire, nous cherchons à déterminer l'ensemble des noeuds en amont de la région post-dominée par V . Nous avons déterminé un algorithme permettant de la définir:

Algorithm 1: Frontière de post-dominance

```
1: procedure POST-DOMINANCE FRONTIER( $V$ )
2:    $\triangleright$  Initialisation de la queue de traitement et des noeuds de l'enveloppe
3:    $res \leftarrow \emptyset$ 
4:    $a\_traiter \leftarrow$  queue contenant les noeuds de  $V$ 
5:    $enveloppe \leftarrow$  ensemble contenant les noeuds de  $V$ 
6:
7:   while  $a\_traiter$  non vide do
8:      $parent \leftarrow "a\_traiter.pop()"$ 
9:     if Tous les successeurs de parents dans  $enveloppe$  then
10:        $enveloppe \leftarrow enveloppe \cup parent$ 
11:        $a\_traiter \leftarrow a\_traiter \cup$  prédecesseurs de  $parent$ 
12:     else
13:        $res \leftarrow res \cup parent$ 
14:     end
15:   end
16:   return  $res$ 
17: end
```

Avec cet algorithme, nous pouvons déterminer la frontière de post-dominance d'un ensemble de noeuds. L'objectif est de pouvoir repérer les deadlocks MPI, et ces outils nous en donnent désormais les moyens. En effet, pour détecter un deadlock, il suffit d'examiner, pour chaque ensemble de collectives MPI de même rang et de même nature, si leur frontière de post-dominance est vide ou si elle ne l'est pas. Si elle ne l'est pas, cela signifie qu'il existe au moins un chemin d'exécution qui n'atteint pas toutes les collectives concernées, ce qui indique un risque de deadlock.

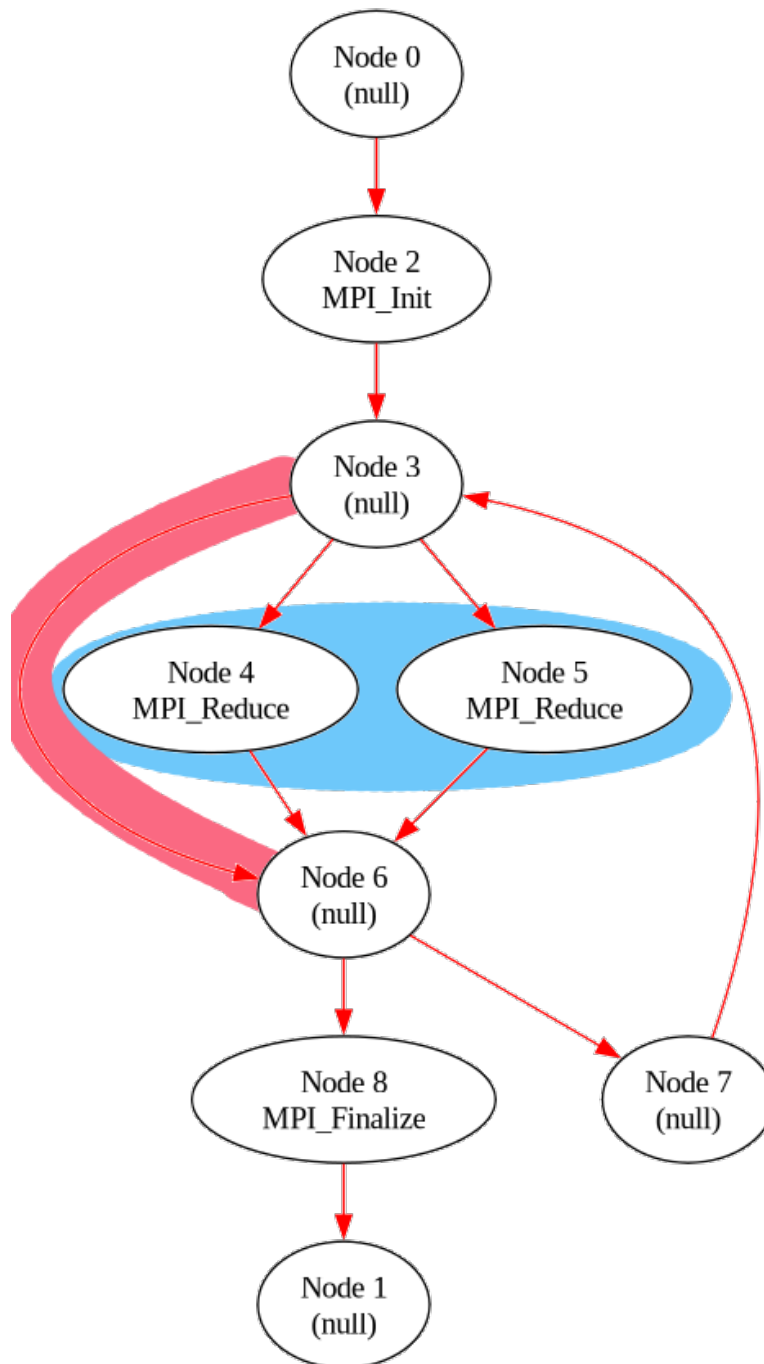


Figure 6: Ici $\text{PDF}(\{4,5\})=\{3\}$ car il existe le chemin en rouge qui n'atteint ni 4 ni 5 mais qui arrive quand même à 6.

II.6 Résultat du plugin

Nous parvenons donc à détecter les deadlocks MPI ainsi nous pouvons afficher un warning lorsqu'un deadlock potentiel est repéré et donner sa localisation dans le code source via la fonction

```

warning_at(loc, OPT_fplugin_,
           "Line %<%d%> can cause an invalid fork when evaluating MPI
collectives",
           gimple_lineno(stmt));

```

```

test2.c: In function 'mpi_call':
test2.c:13:6: warning: Line '13' can cause an invalid fork when evaluating MPI collectives [-fplugin=]
  13 |   if (c < 10) {
      |       ^
[GRAPHVIZ] Generating CFG of function mpi_call in file <mpi_call_test2.c_6__cfg.dot>
test2.c: In function 'main':
test2.c:32:17: warning: Line '32' can cause an invalid fork when evaluating MPI collectives [-fplugin=]
  32 |   for (i = 0; i < 10; i++) {
      |               ^

```

Listing 1: Code qui génère le warning et son exécution

III Conclusion / pistes d'amélioration

En conclusion, nous avons implémenté un plugin GCC permettant d'avertir en cas de possible deadlock MPI. Nous nous sommes rendu compte au cours de ce projets des enjeux des passes de compilation et de manipulation du cfg.

Nous sommes devenus familier avec un ensemble de fonctionnalités proposées par GCC pour le développement de plugin et différents problèmes algorithmiques soulevé par la manipulation de graphe (post-dominance, back-edge, etc...).

Nous avons aussi calculé les frontières de post-dominance itérée mais nous n'avons pas réussi à déterminer de manière satisfaisante des informations supplémentaires pour les avertissements. Nous faisons aussi beaucoup d'appels pour calculer les informations de post-dominance via les fonctionnalités internes de GCC alors qu'il serait possible de faire un seul appel en début de passe.